布丁（pudding）

原题：Luogu P16737。

题面说人话就是求有多少个 $[l,r]$ 中的正整数 a 满足：a 的奇因子数与偶因子数之差的绝对值不超过 2。

考虑记 $a=2^k\times b$，其中 b 是奇数。记 b 的因子数为 d，则 a 有 d 个奇因子与 $k\times d$ 个偶因子。

当 $k=0$ 时，我们要求 $d\le 2$，此时有 $a=1$ 或 a 是奇质数。

当 $k=1$ 时，一定满足要求，此时有 a 模 4 余 2。

当 $k=2$ 时，我们要求 $d\le 2$，此时有 $a=4$ 或 a 是奇质数的 4 倍。

当 $k=3$ 时，我们要求 $d=1$，此时有 $a=8$。

当 $k\ge 4$ 时，一定不满足要求。

那么只需要快速计算 $[l,r]$ 以及 $[\frac{l}{4},\frac{r}{4}]$ 中分别有多少个质数就可以了。为了做到这一点，我们可以先预处理出 $\le\sqrt{r}$ 的所有质数，然后用这些质数对 $[l,r]$ 和 $[\frac{l}{4},\frac{r}{4}]$ 分别跑一遍埃氏筛。

时间复杂度为 $O(t\frac{\sqrt{r}}{\log\sqrt{r}}+\sum((r-l)\log\log(r-l)))$。

子序列（subsequence）

原题：Luogu P6728。

考虑直接 dfs 出字典序前 K 小的子序列。由于可能会有多个值完全相同的子序列，一个 dfs 状态需要记录两个信息：当前 dfs 到的子序列 s（实现时记录 $h(s)$ 即可），以及所有值与 s 完全相同的子序列的结尾位置构成的可重集 $endpos$。

实现时，我们需要精确地从小到大找到所有在序列 A 中最后一次出现的位置 $> \min(endpos)$ 的值，这可以使用倍增实现。对于每一个这样的值 val，为了计算下一层递归中需要的结尾位置集合 $endpos'_{val}$，我们需要精确地取出它在序列 A 中所有 $> \min(endpos)$ 的出现位置，然后对于每一个这样的位置 pos，计算 $endpos$ 集合中 $< pos$ 的位置个数 cnt，并在 $endpos'_{val}$ 中加入 cnt 个 pos，这些过程都是容易使用二分查找等实现的。

如果我们准确地实现了以上过程，那么每找到一个符合条件的子序列，都只需要付出 $O(\log n + \log K)$ 的代价（值域可以离散化，因此认为值域大小 $\leq n$，虽然在本题中不用离散化）。因此总时间复杂度为 $O(n \log n + K(\log n + \log K))$。

传送（teleport）

原题：Luogu P16705。

下文中：

- $\gg$ 的含义同 C++ 中的右移运算符，$\oplus$ 的含义同 C++ 中的按位异或运算符；
- “点 u 所在的第 k 层连通块”指在熟练度为 2^k-1 时，从点 u 经过零次、一次或多次传送，能够到达的点构成的集合。

定义 d_u 为从点 u 到根的路径上所有边的边权异或和。再定义 $to_{u,k}$ 为点 u 及其所有祖先节点中，最浅的满足 $d_x \gg k = d_u \gg k$ 的点 x。可以发现，$to_{u,k}$ 的子树中所有满足 $d_x \gg k = d_u \gg k$ 的点 x 恰好构成了点 u 所在的第 k 层连通块。

对于一次询问 (u,v)，我们找到最小的非负整数 m 使得 $to_{u,m} = to_{v,m}$。特判掉 $m=0$ 的情况，此时答案为 0。其余情况中，答案一定 $\in [2^{m-1}, 2^m)$。进一步地，可以发现：若 u 与 $to_{u,m}$ 不在同一个第 $m-1$ 层连通块中，则从 u 出发不断传送，首次传送到与 u 在同一个第 m 层连通块中，但不在同一个第 $m-1$ 层连通块中的点时，传送到的这个点一定与 $to_{u,m}$ 在同一个第 $m-1$ 层连通块中；对于 v 来说也是一样的。因此，若定义 $g_{u,k}$ 为从点 u 经过若干次传送，到达与点 u 在同一个第 k 层连通块中，但不在同一个第 $k-1$ 层连通块中的点所需的最小熟练度，则答案为 $\max\{g_{to_{u,m-1},m}, g_{to_{v,m-1},m}\}$。

现在考虑如何计算 $g_{u,k}$。定义 $f_{u,k}$ 为最小的满足 x 是 u 的祖先，$d_x \gg k \neq d_u \gg k$，且 $d_x \gg (k+1) = d_u \gg (k+1)$ 的 $d_x \oplus d_u$。那么，$g_{u,k}$ 就等于最小的满足 x 在 u 的子树中的 $f_{x,k-1}$（这里的 x 是枚举了到达不同的第 $k-1$ 层连通块的那次传送的两个端点中，深度较深的一个）。

考虑如何快速计算以上过程中需要的各个值。进行一次 dfs，并同时维护一个包含从当前 dfs 到的点到根的路径上的所有点 x 的 d_x 的 01 trie，即可快速计算 to 数组和 f 数组。求出 f 数组后，求 g 数组就是一些 RMQ 问题，这是容易解决的。回答询问时，直接暴力找 m 即可。

记边权的最大值为 V，则时间复杂度为 $O(n \log V(\log n + \log V) + Q \log V)$。

划分（partition）

原题：Codeforces 1750G。

显然，如果 P' 中有 c 个 $2 \le k \le n$ 满足 $p'_k = p'_{k-1} + 1$，那么 $f(P') = n - c$。

先考虑没有字典序限制时怎么做。考虑容斥，钦定一些值 x 满足：在排列中，x 后面紧接着的值为 $x+1$。如果我们能够计算出数组 f，其中 f_i 表示有 i 个值被钦定满足以上限制时的方案数，则对 f 跑一遍二项式反演即可得出答案。而计算出数组 f 是简单的，因为进行了上述钦定之后，就相当于一些值被粘到一起，形成了若干个段，而段与段之间的顺序可以是任意的。

现在考虑字典序限制。枚举 P' 与 P 的最长公共前缀长度 i，再枚举 p'_{i+1} 的值（必须 $< p_{i+1}$）。可以发现上述的容斥仍然是可以正常进行的，只不过可以被钦定的值少了一些。计算出 g_j 表示：p'_{i+1} 以及剩下尚未被填入的值中，钦定有 j 个值 x 后面紧接着的是 $x+1$ 的方案数。再计算出 c 表示满足 $2 \le k \le i+1$ 且 $p'_k = p'_{k-1} + 1$ 的 k 的个数。对 g 跑一遍二项式反演得到 g'，则 g'_j 就是此时恰好有 $j+c$ 个 $2 \le k \le n$ 且 $p'_k = p'_{k-1} + 1$ 的 k 的方案数。这样做的时间复杂度为 $O(n^4)$。

一个简单的优化是，枚举最长公共前缀长度 i 后，很多 p'_{i+1} 都会计算出完全相同的 (g, c) 二元组。具体地，枚举 i 后，(g, c) 二元组的取值只与 $p'_{i+1} - 1$ 是否在 P 的前 i 个位置中出现，$p'_{i+1} + 1$ 是否在 P 的前 i 个位置中出现，以及是否有 $p'_{i+1} = p_i + 1$ 有关。这意味着枚举 i 后，(g, c) 二元组的种类数是 $O(1)$ 的。因此我们可以在除二项式反演以外的部分做到 $O(n^2)$，然而二项式反演仍然需要进行 $O(n)$ 次，因此时间复杂度为 $O(n^3)$。

刚刚我们是先对于所有 (g, c) 进行二项式反演，再平移后相加。能否先将所有 (g, c) “平移”后相加，再只进行一次二项式反演呢？从组合意义的角度考虑，可以得到，对于一个 (g, c)，只需要对它的已经填好的 c 个满足 $a_k = a_{k-1} + 1$ 的位置，也分别钦定它们是否满足 $p'_k = p'_{k-1} + 1$，就相当于实现了“平移”。此时只需将所有“平移”后的 g 相加，再跑一遍二项式反演即可。

但是为了实现对 g 的“平移”，我们需要进行 c 次对于所有 j，同时令 $g_j \leftarrow g_j + g_{j-1}$ 的操作。因此时间复杂度仍然是 $O(n^3)$ 的。不过我们可以考虑一个秦九韶算法状物。维护一个初值全为 0 的二维数组 h，每次计算出 (g, c) 后，对于所有 j，令 $h_{c,j} \leftarrow h_{c,j} + g_j$。将全部 (g, c) 加入 h 后，从 $n-1$ 到 0 枚举 i，枚举到每个 i 时，对于所有 j，令 $h_{i,j} \leftarrow h_{i,j} + h_{i+1,j} + h_{i+1,j-1}$。结束以上步骤后，$h_0$ 就是所有“平移”后的 g 之和。对 h_0 进行一遍二项式反演即可得出答案。

时间复杂度为 $O(n^2)$。